

Question 1

A city council currently uses separate systems to manage parking permits, waste collection services, and citizen complaints.

These systems operate independently and do not share data. This results in inconsistent records, duplicated information, and repeated manual data entry by staff. Citizens often experience delays when requesting services or tracking updates.

The council plans to develop a unified Citizen Services Platform to improve efficiency and service delivery. The new platform must integrate with the existing systems, comply with UK data protection legislation, and should be delivered within the available budget and project timescale.

a) Define the purpose of systems analysis, then analyse the current problems and identify TWO (2) key stakeholders involved in the proposed system. (5 marks)

The purpose of system analysis is to understand the existing system, identify its problems, and define user requirements for the system.

After analyzing,

1. Separate systems operate independently and do not share data.
2. Results in duplicated information and conflicting records.

Two key stakeholders,

1. **Citizens (End Users):** People who request services (parking permits, waste collection) and track updates.
2. **Council Staff (Internal Users):** Employees who manage services, handle citizen complaints, and currently perform manual data entry.

b) Define the system scope and identify THREE (3) constraints that should be considered during the development of the proposed platform. Use examples from the scenario. (5 marks)

The system scope is the **boundaries of the project**, defining what the new **Unified Citizen Services Platform** *will* include (integrating parking, waste collection, and complaints) and *will not* include.

THREE (3) constraints,

1. **Legal Constraint:** Must comply with **UK data protection legislation** when handling citizen information.
2. **Financial Constraint:** Must be delivered strictly **within the available budget**.
3. **Time Constraint:** Must be completed and deployed **within the specified project timescale**.

Question 2

A gym currently manages class bookings manually at the reception desk. Members must ask staff to check availability, and bookings are recorded on paper or spreadsheets. This often leads to double bookings, missed sessions, and delays in updating membership information.

The gym plans to develop a mobile application that allows members to manage class bookings and memberships more efficiently.

a) Define functional and non-functional requirements. Then identify TWO (2) examples of each from the scenario. (5 marks)

Functional Requirement: What the system must do (its specific behaviors, features, and functions).

Non-Functional Requirement: How the system performs (its quality attributes, constraints, performance, and security).

TWO (2) Functional Examples,

1. Allow members to **check class availability** via the mobile app.
2. Allow members to manage/book class bookings and memberships digitally.

TWO (2) Non-Functional Examples,

1. **Reliability:** The system must prevent double bookings and missed sessions that currently happen with paper/spreadsheets.
2. **Usability:** The mobile application must provide fast updates to eliminate reception desk delays in updating membership information.

b) Explain the purpose of the MoSCoW prioritisation technique. Then give THREE (3) suitable examples from the gym mobile application, using three different MoSCoW categories. (5 marks)

The purpose of the MoSCoW is to prioritize requirements into categories (**Must have, Should have, Could have, Won't have**) so the project team knows what features are critical for delivery within budget and time.

THREE (3) Examples,

1. **Must Have (M):** Feature vital for the system to work. E.g., *Member class booking and checking availability.*
2. **Should Have (S):** Important but not vital for day-one launch. E.g., *Push notifications to remind members of upcoming booked gym sessions.*
3. **Could Have (C):** Desirable if time and budget permit. E.g., *In-app chat support with gym reception staff.*

Question 3

- a) Define abstraction and inheritance. Provide suitable example of each and explain the difference between the two object-oriented principles. (5 marks)

Abstraction is hiding complex implementation details and showing only the essential features/behavior to the user.

Example,

An online **Shopping Cart** has a checkout() button, The user just presses it without needing to understand complex internal background tasks like calculating tax, updating inventory databases, and processing payment gateways.

Inheritance is a mechanism where a class (child/subclass) can acquire the properties and methods of another class (parent/superclass) to promote code reusability.

Example,

An **Employee** parent class has attributes like name and salary. A **Manager** child class inherits these attributes and adds its own specific attribute like teamSize.

Abstraction focuses on *what* an object does (external view/interface), whereas **Inheritance** focuses on *code sharing and hierarchical relationships* between classes.

b) Define encapsulation and information hiding. Provide a suitable example of each and explain how these principles help protect object data and reduce dependency between system components. (5 marks)

Encapsulation is wrapping data (attributes) and code (methods) together into a single unit (class).

Information Hiding is restricting direct access to an object's internal data using visibility modifiers (like private or protected).

Example: A **UserAccount** class keeps the password attribute private, requiring external classes to interact through public methods like setPassword() or login().

How it protects the system,

Prevents unauthorized external classes from making direct, invalid changes to internal object states, ensuring data integrity.

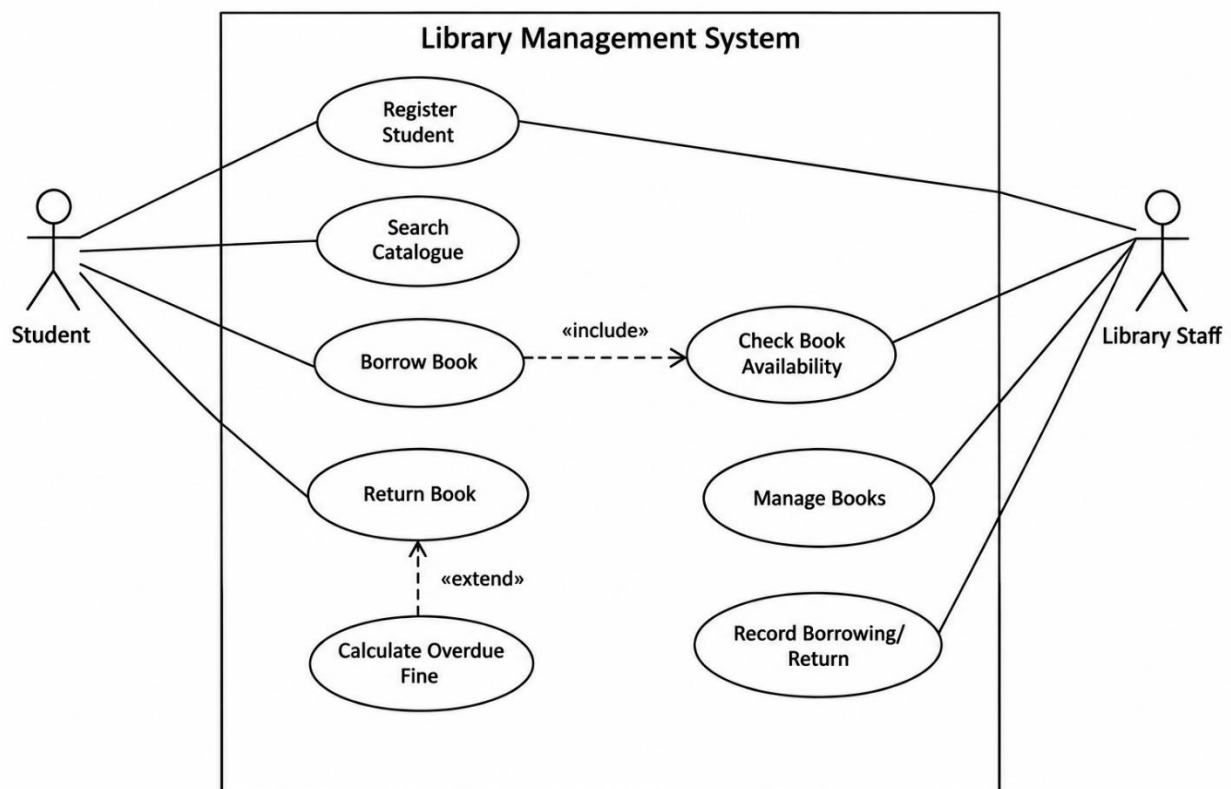
Components only interact through well-defined public interfaces, meaning internal code changes inside a class won't break other connected system components.

Question 4 (10 marks)

A university library currently uses a manual process for managing books. Students must register with the library before borrowing books. Library staff manually records borrowing and returns, check book availability, and calculate overdue fines for late returns. Books are often misplaced, fines are calculated inconsistently, and there is no central system for searching the catalogue or tracking loans. The university plans to introduce a Library Management System to improve these processes.

Draw a UML Use Case Diagram for the proposed system, using correct UML notation. Your diagram should:

- identify the actors and use cases,
- include a labelled system boundary,
- use appropriate associations,
- include ONE (1) «include» relationship and ONE (1) «extend» relationship where appropriate.



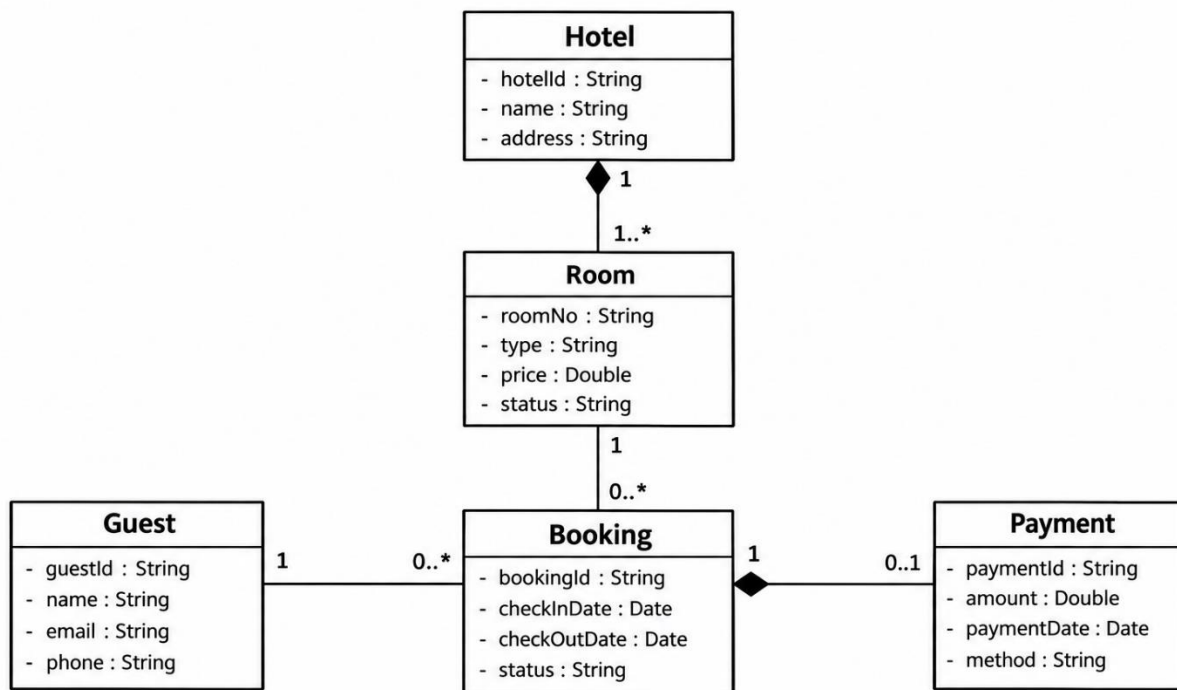
Question 5 (10 marks)

A Hotel Reservation System manages information about hotels, rooms, guests, bookings, and payments. Each hotel contains multiple rooms. Each booking is made by one guest for one room, and a guest may make multiple bookings. A booking may generate one payment, but a payment cannot exist without its associated booking.

Draw a UML Class Diagram for the proposed Hotel Reservation System using correct UML notation. Your diagram should:

- identify the appropriate classes and attributes,
- show the relationships between classes,
- include appropriate multiplicities,
- identify where composition should be used.

Hotel Reservation System

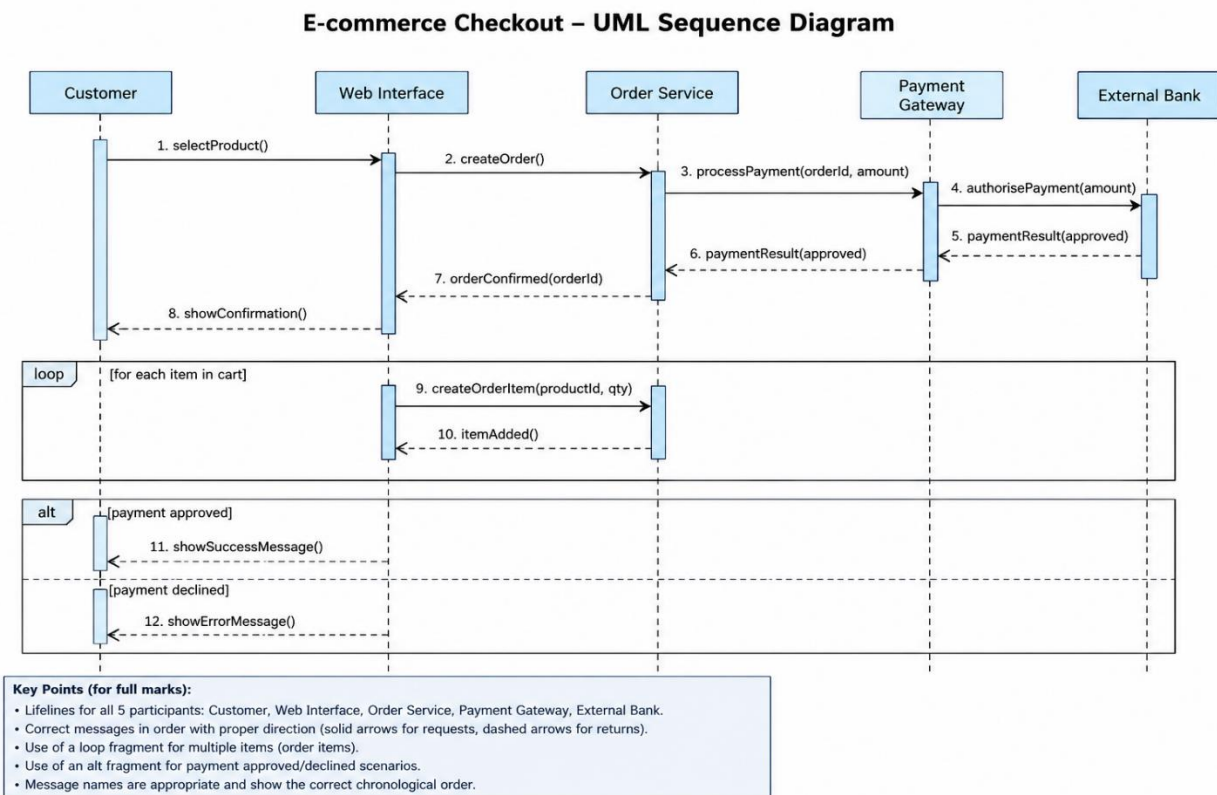


Question 6 (10 marks)

An e-commerce system allows customers to purchase products online. During checkout, the Customer submits an order through the Web Interface. The Order Service validates the order and sends a payment request to an external Payment Gateway. Once payment is authorised, the Order Service confirms the order, and the Web Interface displays the confirmation to the customer.

Draw a UML Sequence Diagram for the checkout process using correct UML notation. Your diagram should:

- Include the lifelines identified in the scenario
- Show the messages exchanged between the lifelines
- Show the messages in the correct chronological order



Question 7

A development team is preparing a system for deployment. They must ensure that the system is correctly built and meets user requirements before release.

a) Define validation and verification with ONE (1) example of each. (5 marks)

Verification: Checking that the software is built correctly according to the design specifications ("Are we building the product *right*?").

Example,

Developers review source code line-by-line against programming standards to find syntax or design errors before testing.

Validation: Checking that the software actually meets the user's real needs and business requirements ("Are we building the *right* product?").

Example,

End-users test the deployed system in a staging environment to confirm it solves their actual business problems.

b) Identify and explain the purpose of any THREE (3) testing techniques used to ensure software quality.
(5 marks)

1. Unit Testing

Testing individual components, functions, or classes in isolation to ensure each unit works correctly on its own (usually performed by developers).

2. Integration Testing

Testing how multiple integrated units or modules work together to ensure data flows correctly between them without interface errors.

3. System Testing

Testing the complete, fully integrated software system against both functional and non-functional requirements before release to the client.

Question 8

a) Clear and well-defined requirements are essential for successful system development. Identify THREE (3) benefits of clearly defined requirements and describe TWO (2) practices used during development to ensure requirements are correctly implemented. (5 marks)

- 1. Prevents Scope Creep:** Keeps the project focused by clearly stating what is in and out of scope, avoiding unauthorized feature additions.
- 2. Reduces Costs & Rework:** Catching and fixing requirement errors early is much cheaper than fixing them during coding or after deployment.
- 3. Better Communication:** Provides a shared, unambiguous understanding between stakeholders, clients, and the development team.

Two Practices,

- 1. Requirements Traceability Matrix (RTM):** A table that maps each user requirement to its corresponding design element, code component, and test case to ensure nothing is missed.
- 2. Formal Reviews and Walkthroughs:** Conducting regular review meetings with clients and stakeholders to sign off on requirements before development begins.

b) Other than “functional” and “non-functional” requirements, give FIVE (5) types of considerations that should be recorded when documenting system requirements. (5 marks)

1. Legal Considerations: Compliance requirements such as data protection laws, copyright, or industry-specific regulations.

2. Operational Considerations: Where and how the system will run, including hardware/software compatibility and operating environments.

3. Security and Privacy Considerations: Data encryption, user access control policies, and authentication mechanisms needed to protect the system.

4. Interface and Integration Considerations: How the new system will communicate and exchange data with external third-party systems or legacy software.

5. Project Constraints (Budget and Time): Financial limits, resource availability, and critical delivery deadlines that impact how requirements are met.

1. Requirements Engineering & Management

- **Requirements Workshop:** A structured, collaborative meeting where stakeholders, users, and developers come together to negotiate, prioritize, and resolve conflicting requirements.
- **Software Requirements Specification (SRS):** The official, formal document that defines all functional and non-functional requirements of the system.
- **Requirements Traceability:** The ability to trace a requirement backward and forward through its lifecycle (from initial user need → design specification → source code → test case).

2. Object-Oriented (OO) Principles

- **Polymorphism:** The ability of different subclasses to implement the same inherited method in their own unique way (e.g., a superclass `Employee` has `calculateSalary()`, but `Manager` and `Intern` compute it differently).
- **Encapsulation:** Wrapping data (attributes) and methods into a single unit (class) while shielding internal states.
- **Information Hiding:** Restricting direct external access to an object's internal data (using private fields and public getters/setters) to protect data integrity.

3. Class Relationships (UML Multiplicities & Associations)

- **Multiplicity** `0..*`: Means "zero or many" (optional and can have multiple instances). Other common ones: `1` (exactly one), `1..*` (one or many), `0..1` (zero or one).
- **Aggregation** ( **Open Diamond**): A "has-a" weak relationship where the child object can exist independently of the parent (e.g., a university and a student, or a module and a student). If the parent is destroyed, the child survives.
- **Composition** ( **Filled Diamond**): A "has-a" strong relationship where the child object cannot exist independently of the parent (e.g., a room belongs to a hotel, or a payment belongs to a booking). If the parent is destroyed, the child dies with it.
- **Generalization:** An "is-a" inheritance relationship between a general parent class and a specialized child class (shown with a hollow triangle arrow).

4. Software Quality & Architecture

- **High Cohesion:** When a class or module focuses on performing one single, well-defined task clearly. (Goal: High cohesion is *good*).
- **Low Coupling:** When system modules have minimal dependencies on one another, meaning changes in one class won't easily break another. (Goal: Low coupling is *good*).

5. UML Diagrams

- **Activity Diagram:** Best for modeling step-by-step business workflows, processes, parallel processing, and decision-making branches (using diamond decision nodes).
- **Sequence Diagram:** Focuses on the chronological order of message exchanges between lifelines over time.
- **Use Case Diagram:** Shows actors, system boundaries, and system functionalities (use cases) to capture user requirements.

6. Testing Types

- **Unit Testing:** Testing individual methods, classes, or components in isolation (done by developers).
- **Integration Testing:** Testing how multiple modules or units work together when combined.
- **Regression Testing:** Re-running existing tests after code modifications to verify that recent changes haven't broken previously working features.
- **Acceptance Testing (UAT):** Testing performed by end-users to confirm the system fulfills business needs before final deployment.